Fases, objetivos y estado actual TFM

1. Objetivos que cumplir

Se han establecido 9 objetivos iniciales que cumplir en el proyecto, de manera que podamos construir un proyecto sólido que cumpla con todos los objetivos del TFM de nuestro Máster.

- **Construir la capa de ingesta de eventos**: Implementar un productor capaz de transformar registros del dataset de ciberseguridad en eventos estructurados y publicarlos continuamente en un topic de Apache Kafka.
- **Simular tráfico de red en tiempo real**: Reproducir registros históricos de CICIDS2017 respetando, cuando exista, la información temporal original y permitiendo acelerar o controlar la velocidad de reproducción para crear diferentes cargas de trabajo.
- **Establecer un esquema de evento reutilizable**: Definir un formato de mensaje consistente que permita transportar los datos necesarios para las siguientes etapas del pipeline, incluyendo timestamp, protocolo, puerto de destino, características del flujo, etiqueta de referencia y metadatos del dataset.
- **Incorporar Apache Flink al flujo**: Desarrollar un job de procesamiento en streaming que consuma los eventos de Kafka y permita realizar transformaciones, ventanas y operaciones sobre el flujo de forma continua.
- **Integrar la inferencia de Machine Learning**: Integrar un modelo de clasificación previamente entrenado para que la predicción se realice sobre los eventos a medida que atraviesan el pipeline, evitando convertir el proceso de detección en una ejecución batch independiente.
- **Generar resultados y alertas operativas**: Producir una salida estructurada con la predicción, información temporal y datos relevantes del evento para que pueda ser almacenada y posteriormente visualizada.
- **Construir la capa de monitorización**: Desarrollar un dashboard en Grafana que permita observar en tiempo real el volumen de eventos, las detecciones y los principales tipos de ataque, junto con métricas de funcionamiento del pipeline.
- **Evaluar el rendimiento del pipeline**: Medir la latencia extremo a extremo y el throughput, así como estudiar el comportamiento del sistema ante incrementos controlados de la tasa de eventos y posibles situaciones de saturación o backpressure.
- **Evaluar la eficacia del clasificador**: Comparar las predicciones realizadas en streaming con las etiquetas de referencia para obtener matriz de confusión,

Accuracy, Precision, Recall y F1-Score, prestando especial atención a los falsos positivos.

2. Fases del proyecto

Inicialmente el proyecto se ha estructurado en 6 fases. Actualmente nos encontramos en la fase inicial, la cual consiste en generar la de Ingesta y simulación, ya que de esta fase depende el desarrollo de todas las demás.

Fase	Objetivo
Fase 1: Ingesta y simulación	Consolidar el productor y consumidor Kafka, validar el esquema de evento y reproducir CICIDS2017 a diferentes velocidades.
Fase 2: Pipeline Kafka → Flink	Crear el job básico de Flink, consumir eventos, realizar transformaciones/ventanas y validar el procesamiento extremo a extremo.
Fase 3: ML en streaming	Integrar el modelo entrenado, ejecutar inferencia evento a evento y generar resultados etiquetados.
Fase 4: Persistencia y Grafana	Almacenar los resultados y construir el dashboard de alertas y métricas.
Fase 5: Evaluación	Ejecutar pruebas de carga, medir latencia y throughput, analizar pérdida de datos/backpressure y calcular métricas de clasificación.
Fase 6: Integración y conclusiones	Validar la arquitectura completa, documentar resultados y establecer las conclusiones respecto a los objetivos y preguntas de investigación.

3. Estado actual

El primer paso a realizar en nuestro proyecto ha sido construir un productor que reproduzca los datos de CICIDS2017 hacia Kafka a ritmo controlado, simulando tráfico en vivo.

Para ello se ha construido un entorno local con Docker Compose para poder ejecutar Kafka localmente.

Para poder consumir de Kafka, se han creado dos ficheros de Python de consumidor y productor.

En la siguiente tabla veremos las características implementadas hasta al momento

Componente	Estado actual	Función
Kafka	Implementado	Broker único ejecutado mediante Docker Compose, en modo KRaft, accesible por localhost:9092.
Topic	Implementado	Topic de trabajo: network-events.
Producer	Implementado	Lee CICIDS2017, construye eventos JSON y los publica en Kafka.
Control temporal	Implementado	Usa Timestamp real si está disponible y permite acelerar la reproducción mediante --speed.
Control de carga	Implementado	Permite limitar registros (--limit), saltar registros (--skip) y establecer intervalos máximos/fijos.
Consumer	Implementado	Consume los eventos y muestra en vivo el total recibido y el número de ataques.

4. Futuros pasos

En las futuras entregas iremos desarrollando las características pendientes progresivamente, de manera que iremos incorporando las herramientas de una en una para asegurarnos que la conexión entre ellas funciona de manera correcta.

Flink	Pendiente	Consumirá el topic y realizará el procesamiento continuo.
Machine Learning	Pendiente	Se integrará para realizar la clasificación de cada evento en streaming.
Persistencia / sink	Pendiente	Se definirá para almacenar resultados y métricas.
Grafana	Pendiente	Visualizará alertas, volumen de eventos y métricas del sistema.